AIO 2022 — Problem 5: Composing Pyramids — Full Solution

1. Restating the problem in algorithmic terms

A **musical pyramid** is a sequence `v, v+1, ..., m, m-1, ..., v+1, v` for some base value `v` and peak value `m ≥ v`. Its length is `2*(m-v) + 1` — always odd, as required. The special case `v = m` is just a single key, length 1.

We're asked: what's the longest **subsequence** of `P` (order preserved, not necessarily contiguous) that forms a musical pyramid? The answer to the problem is `N - (that longest length)`.

So this is a longest-subsequence-with-shape problem, similar in spirit to Longest Increasing Subsequence, but the "shape" is: strictly-by-1 ascending run, then strictly-by-1 descending run, symmetric around a peak.

2. Key insight: split at the peak

Any valid pyramid has exactly one peak position (wherever the value `m` sits). So instead of searching over all `(v, m)` pairs directly, fix the **position** `i` that will serve as the peak, and ask two independent questions:

- **Looking left/up to position `i`:** what's the longest run `v, v+1, ..., P[i]` I can pick as a subsequence of `P[1..i]` that *ends* at position `i`?
- **Looking right/down from position `i`:** what's the longest run `P[i], P[i]-1, ..., v'` I can pick as a subsequence of `P[i..N]` that *starts* at position `i`?

Call these `up[i]` and `down[i]` (counting positions, so a run of `k` consecutive integers has `up[i] = k`, not `k-1`).

Why this decomposes cleanly: the ascending half only depends on what's *before* `i`, and the descending half only depends on what's *after* `i` — they share nothing except the peak itself, so they can be computed completely independently and then combined.

3. Why the "suffix" property holds — and why we can just take

a min

Crucial sub-claim: if `up[i] = k` (i.e. there's some subsequence ending at `i` covering `P[i]-k+1, ..., P[i]`), then for **every** `r` from `0` to `k-1`, a radius-`r` ascending run ending at `i` also exists. This is true because the witnessing subsequence's own *suffix* (its last `r+1` terms) is itself a valid shorter ascending run ending at `i` — no extra search needed, it's free. The same holds for `down[i]` on the descending side.

This matters because a real pyramid needs **both sides to reach the same base value `v`**. Since any radius up to `up[i]-1` is achievable ascending, and any radius up to `down[i]-1` is achievable descending, the largest radius achievable on *both* sides simultaneously is simply:

```
radius(i) = min(up[i] - 1, down[i] - 1)
pyramidLength(i) = 2 * radius(i) + 1
```

Maximize `pyramidLength(i)` over all `i`, and that's the longest pyramid in the whole array.

4. Computing `up[]` and `down[]` in O(N)

This is the classic "longest chain of consecutive values" trick, normally seen for LIS-style problems but here using *equality with a -1/+1 step* instead of "any larger value":

- Scan left to right. Keep an array `bestUp[v]` = the best `up[]` value achieved **so far** (i.e. among positions already processed) by any position whose value is `v`.
- At position `i` with value `x = P[i]`: `up[i] = 1 + bestUp[x-1]` (extend the best chain that ended one-less than me), then update `bestUp[x] = max(bestUp[x], up[i])`.

`down[]` is the mirror image: scan right to left, maintain `bestDown[v]`, and `down[i] = 1 + bestDown[x-1]`.

Both passes are O(N) time (plus O(maxValue) to allocate/clear the lookup arrays).

5. Correctness check against the samples

Sample 1: `P = [2,1,3,4,2,1]` `up = [1,1,2,3,2,1]`, `down = [2,1,3,1,2,1]`. Best peak is position 3 (value 3): `radius = min(2-1, 3-1) = 1` → length 3 → pyramid `[2,3,2]`. Answer = `6 - 3 = 3`. ✅ matches.

Sample 2: `P = [10,11,12,11,10]` `up = [1,2,3,2,1]`, `down = [1,2,3,2,1]`. Peak at position 3 (value 12): `radius = min(2,2) = 2` → length 5 (the whole array). Answer = `5 -`

`5 = 0`. ✅ matches.

Sample 3: `P = [8,3,5,7]` `up = [1,1,1,1]`, `down = [2,1,1,1]`. Best radius anywhere is 0 → length 1. Answer = `4 - 1 = 3`. ✅ matches.

6. Complexity

- Time: **O(N + V)**, where `V = 100,000` is the max key value — one left-to-right pass, one right-to-left pass, one combining pass.
- Memory: **O(N + V)** — well inside 1 GB.

This handles all five subtasks at once (Subtasks 3 and 4 are really just special cases that hint toward this general idea — Subtask 3's "base is always 1" nudges you toward thinking about a fixed endpoint, and Subtask 4's prefix structure nudges you toward the chain-DP).

7. Full commented C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int N;
    scanf("%d", &N);

    vector<int> P(N + 1); // 1-indexed
    for (int i = 1; i <= N; i++) scanf("%d", &P[i]);

    const int MAXV = 100001; // key values are 1..100000, so valid array indices

    // ---------- Pass 1: up[i] ----------
    // up[i] = length of the longest run of consecutive ascending integers
    //         (v, v+1, ..., P[i]) that can be picked as a SUBSEQUENCE of P[1..i]
    //         ending exactly at position i.
    //
    // bestUp[v] = the best up[] value seen so far (among positions already
    //             processed, i.e. to the left of the current one) for value v.
    vector<int> up(N + 1);
    vector<int> bestUp(MAXV, 0);

    for (int i = 1; i <= N; i++) {
        int x = P[i];
        // Extend whatever chain ended one-less than x. If nothing ended at x-1
        // yet, bestUp[x-1] is 0, so up[i] just becomes 1 (a fresh chain of lengt
        up[i] = 1 + bestUp[x - 1];
```

```
        bestUp[x] = max(bestUp[x], up[i]);
    }

    // ---------- Pass 2: down[i] (mirror image, scanned right to left) ---------
    // down[i] = length of the longest run of consecutive descending integers
    //           (P[i], P[i]-1, ..., v) that can be picked as a SUBSEQUENCE of
    //           P[i..N], starting exactly at position i.
    vector<int> down(N + 1);
    vector<int> bestDown(MAXV, 0);

    for (int i = N; i >= 1; i--) {
        int x = P[i];
        down[i] = 1 + bestDown[x - 1];
        bestDown[x] = max(bestDown[x], down[i]);
    }

    // ---------- Combine: try every position as the PEAK ----------
    // Both sides of the pyramid must reach the SAME base value, so the radius
    // achievable through position i is capped by whichever side is weaker.
    int bestLen = 1; // a single key alone is always a valid pyramid (radius 0)
    for (int i = 1; i <= N; i++) {
        int radius = min(up[i] - 1, down[i] - 1);
        bestLen = max(bestLen, 2 * radius + 1);
    }

    printf("%d\n", N - bestLen);
    return 0;
}
```

8. A note on the bug-prone spot (worth flagging for students)

The single place this solution is easy to get subtly wrong is conflating `up[i]` (a *count of positions*) with the *radius* (a count of steps). The radius is always `up[i] - 1` and `down[i] - 1`, **not** `up[i]` and `down[i]` directly — off-by-one here silently gives wrong answers on cases where the peak itself contributes asymmetrically. Sample 3 above is a good one to hand to students to catch this: if you forget the `-1`'s, you still get the right *answer* by coincidence (everything's radius 0 there) — Sample 1 or 2 will actually expose the bug.